```
  def up
    @task.move_up
    redirect_to(@task.project)
  end

  def down
    @task.move_down
    redirect_to(@task.project)
  end

  private

  def load_task
    @task = Task.find(params[:id])
  end

  def task_params
    params.require(:task).permit(:project_id, :title, :size)
  end
end
```

And … now the integration test passes, all the tests are green, and you celebrate.

Retrospective

Let's take a step back and discuss what happened here. You're supposed to be learning about integration tests, and yet you never touched the integration test after you initially wrote it. What good is that? (Full disclosure: in initial writing I did have to go back and clean up the integration test because I had syntax errors and the like. Sometimes I need to fix the assertions after I see what the view looks like.)

The integration test offers a few benefits as you write these tests:

- There's a structure; you know what needs to be done next. This is valuable, although given the nature of a lot of server-side development there does tend to be one large gap where the integration test provides the expected outcome but doesn't give much guidance on how to get there.
- The integration test proves that all of your unit tests work together as a cohesive whole (though you really need to test in a browser when you're done). One danger cited for unit tests is that it's easy to see the trees and not the forest. Having an end-to-end test forces you to describe the forest.
- Although the integration test is slow compared to unit tests, it's often lightning-fast compared to manually setting up the browser integration. If you've ever tested a form with dozens of required fields, you can see the benefit here.